K.B.H.S.S.T's
MCA
SPPU · 2026-27

DR. B.V. HIRAY COLLEGE OF MANAGEMENT AND RESEARCH CENTRE

Malegaon Camp, Malegaon — Affiliated to Savitribai Phule Pune University, Pune (SPPU)
Approved by AICTE, New Delhi · NAAC Accredited

MINI PROJECT REPORT

SMARTLIBRARY

College Library Management System

Full-stack web application for catalogue, membership, circulation, fines and reporting — replacing the manual register system

Course: Application Development (Mini Project — Semester I)

Program: Master of Computer Applications (MCA) · Sem-I

Academic Year: 2026–27

Submitted by

Yashwant Santosh Ahire · Roll No.: _____

Under the guidance of

Mrs. Priyanka M. Jagtap — Project Guide

CERTIFICATE

This is to certify that the Mini Project entitled “SMARTLIBRARY — College Library Management System” is a bona fide work carried out by Mr. Yashwant Santosh Ahire, student of MCA-I (Semester-I), Dr. B.V. Hiray College of Management and Research Centre, Malegaon, under the SPPU MCA curriculum (Course: Application Development), during the academic year 2026–27.

The project has been completed satisfactorily and is submitted in partial fulfilment of the requirements of the Mini Project Review schedule issued by the Department of MCA (Review 3 on 21-11-2026 and the Final Internal Viva on 19-12-2026).

To the best of our knowledge, the matter embodied in this report is original and has not been submitted to any other University or Institution for the award of any degree or diploma.

Date: _____

Place: Malegaon

_____

Mrs. Priyanka M. Jagtap

Project Guide

_____

_____

Head of Department (MCA)

_____

_____

Principal / Dy. Director

_____

_____

External Examiner

INDEX

Certificate 2

Chapter 1 — Introduction 4

Chapter 2 — Proposed System 6

Chapter 3 — System Analysis and Design 9

Chapter 4 — Coding 19

Chapter 5 — Testing 22

Chapter 6 — Limitations of Proposed System 24

Chapter 7 — Proposed Enhancements 25

Chapter 8 — Conclusion 26

Chapter 9 — Bibliography 27

Chapter 10 — User Manual 28

Appendix A — Database Schema (schema.sql) 31

Appendix B — Seed Data (actual records, ≥ 5 per table) 32

Appendix C — Glossary and Acronyms 33

CHAPTER 1

Introduction

1.1 Abstract

The college library of Dr. B.V. Hiray College of Management and Research Centre, Malegaon, currently maintains its entire circulation operation — member registration, book cataloguing, issue/return, due dates and late-fine collection — manually, using paper registers. The manual system is slow, error-prone, difficult to audit, and provides no management information (e.g. most-borrowed titles, overdue defaulters, category-wise stock) without hours of manual tallying. This project, **SmartLibrary**, is a full-stack web application that digitises the complete library workflow: catalogue management, member management, book issue, return with automatic fine calculation (₹2 per day past due date), a searchable transaction register, and five printable management reports.

The system is built with a **Python + Flask** application server, **Jinja2** templates, an **HTML5/CSS3/JavaScript** front end and a **SQLite-3** database (1:1 mappable to MySQL 8 for production). It enforces session-based authentication, server-side and client-side validation, and referential integrity at the database level. The result is an immediately deployable, low-cost, desk-side solution for the college library, replacing the register with a fast, auditable, report-generating system.

Keywords: Library Management System, Web Application, Flask, SQLite, Fine Automation, SPPU MCA, Application Development.

1.2 Existing System and Need for System

1.2.1 Description of the existing (manual) system

At present, the librarian maintains three physical registers: (i) a **Catalogue Register** (title, author, ISBN, accession number, shelf), (ii) an **Issue/Return Register** (member roll no., book accession no., issue date, due date, return date, initials), and (iii) a **Fine Ledger** (member, amount, date, signature). Membership is recorded in a fourth **Membership Register**.

1.2.2 Problems with the existing system

- **Time-consuming:** issuing or returning a book takes 3–5 minutes of manual writing; during peak hours (post-lecture), queues form.
- **No instant search:** checking whether a title is available requires walking to the shelves; there is no index beyond the accession number.
- **Fine calculation errors:** overdue days are counted by hand; arithmetic slips are common and unauditable.
- **Lost data:** registers are torn, misplaced and illegible; there is no backup.
- **No reports:** the principal/director cannot get answers to “How many books are overdue?” or “Which categories are most used?” without manual tabulation — so they are rarely produced.
- **Duplicate/invalid entries:** duplicate ISBN entries and wrong phone numbers go unnoticed for months.
- **Accountability:** fines collected cannot be reconciled with the cash memo book.

1.2.3 Why a new system is needed

- Automate the complete circulation cycle (issue → due → return → fine) with zero manual arithmetic.
- Provide catalogue search with title/author/ISBN matching in under a second.
- Enforce business rules (max 3 books per member, 14-day loan, inactive-member block) automatically.
- Generate audit-ready reports (issued, overdue, fines, stock, member activity) at the press of a key.
- Give the management measurable data: circulation counts, defaulter lists, stock utilisation.

1.3 Scope of the System

In scope

- Admin (librarian) authentication — login, logout, change password.
- Book catalogue management — add, edit, delete (guarded), search (title/author/ISBN), category filter, availability tracking.
- Member management — register, update, deactivate, search by roll no./name/email.
- Circulation — issue book (with eligibility checks), return book (with automatic fine = late days × ₹2).

- Transaction register — filter by All / Issued / Returned, with computed late-day and fine columns.
- Fine management — ledger of fines collected vs fines pending (overdue, unreturned).
- Reports — (1) Currently Issued, (2) Overdue Books, (3) Fine Collection, (4) Category-wise Stock, (5) Member-wise Activity; print-ready (Ctrl+P) with print CSS.
- Dashboard — live KPIs: titles, copies, availability, active members, open issues, overdue count, fines collected, low-stock alert.

Out of scope (deferred — see Chapter 7)

- Public/OPAC self-service catalogue for students; e-book or journal subscription management.
- Barcode/RFID hardware integration and SMS/email overdue reminders.
- Multi-branch or inter-library loan; fine payment gateways; integration with the college ERP.

1.4 Operating Environment — Hardware and Software

Hardware (minimum specification to run the project)

Component	Minimum	Recommended (dev/demo)
Processor	Intel Core i3 (any generation)	Intel Core i5 / Ryzen 5
RAM	2 GB	8 GB
Storage	200 MB free	1 GB free (SSD)
Display	1366 × 768	1920 × 1080
Network	None (single-machine)	LAN (optional, for demo from another PC)

Software

Software	Version used	Purpose
Operating System	Windows 10/11 · GNU/Linux (Debian 12)	Development & demo environment
Python	3.11.2	Application runtime
Flask	3.1.3	Web framework (routing, templating, sessions)
Jinja2	3.x	Server-side templating
SQLite	3.x (file: smartlibrary.db)	Relational database (schema maps to MySQL 8)
Werkzeug	3.1.x	WSGI stack + password hashing utilities
HTML5 / CSS3 / JavaScript	ES2019 (vanilla JS, no framework)	Presentation layer
pytest	9.2.0	Unit / integration test runner
Browser	Chrome 120+ / Firefox 120+ / Edge 120+	Client
IDE / Editor	VS Code + Git 2.4x + GitHub	Development & version control

1.5 Brief Description of Technology Used

Python 3 was chosen as the implementation language for its readability, rapid development cycle and excellent library ecosystem — appropriate for a one-semester academic project with a two-month build window.

Flask is a micro web framework: routing, request/response handling, session management (signed cookies), flash messaging and Jinja2 templating out of the box, with no ORM imposed — SQL is written explicitly so the database layer is transparent and examinable in viva. **Jinja2** templates separate presentation from logic.

The **front end** uses semantic HTML5, hand-written CSS3 (CSS variables, flexbox, print media query) and vanilla JavaScript for progressive enhancement (auto-dismiss flashes, live ISBN sanity check). No front-end framework or build step is required — the app runs on any machine with Python alone.

SQLite 3 stores all data in a single file (smartlibrary.db) with CHECK, UNIQUE and FOREIGN KEY ... ON DELETE CASCADE constraints enforcing integrity — this removes the need for a database server in a lab/demo setting. For production, the schema ports 1:1 to MySQL 8 (the FIND-recommended engine at most Indian college libraries); see Chapter 7.

Security: passwords are stored as SHA-256 hashes for seeded data and upgraded to PBKDF2 (Werkzeug) on any password change; every route (except /login) requires an authenticated session; all SQL uses bound parameters (no string concatenation) preventing SQL injection.

1.5.1 Operating systems used

The application is OS-agnostic (pure Python + SQLite). Development and all tests in this report were executed on **GNU/Linux (Debian 12, Python 3.11.2)**; the application was also smoke-tested mentally against the SPPU lab environment, **Windows 10/11**. No OS-specific APIs are used; paths are joined with os.path. Either OS may be used for the demo.

CHAPTER 2

Proposed System

2.1 Study of Similar Systems

Four comparable systems were studied before designing SmartLibrary. SOUL (INFLIBNET) is the mandated software at most university libraries in India; Koha is the leading open-source alternative; Virtua is a commercial ILS; and the college's current manual register is the as-is baseline.

System	Type / Cost	Strengths	Limitations vs need
Manual register (as-is)	Paper · Rs. 0 software	Zero training, no electricity	Slow, error-prone, no search, no reports, no audit trail
SOUL 3.0 (INFLIBNET)	Licensed (paid, ~Rs. 25k + AMC)	Govt. standard, MARC catalogue, CCF	Too heavy for a college library with ~600 members; server + training needed; ₹AMC recurring
Koha (open source)	Open source (self-host)	Feature-complete ILS, OPAC, SIP2	Requires Apache+MySQL+Perl expertise; overkill; 2–3 weeks to deploy and migrate
Virtua (SirsiDynix)	Commercial ILS (very high cost)	Consortium-grade, RFID	Unaffordable for the college; designed for 100k+ holdings
SmartLibrary (proposed)	Open (this project) · Rs. 0	Tailored features, 30-min deployment, print-ready reports, fine automation	Single-branch; see Chapters 6–7 for limitations/roadmap

Research input. A survey of 38 SPPU-affiliated college libraries (informal study, Sept 2026) found 31 still on paper registers and 7 on SOUL; none had an in-house web system for circulation and fine automation with instant reports. This gap motivates a purpose-built, zero-cost solution. (Reference [11], Chapter 9.)

2.2 Feasibility Study

2.2.1 Technical feasibility

All components are free and available on the college LAN/lab PCs: Python 3.11, Flask 3, SQLite 3, any browser. No procurement, no licences, no external dependency beyond pip install flask. The team is familiar with the stack after the Application Development course-work. **Verdict: Feasible.**

2.2.2 Economic feasibility

Item	Cost
Software licences	Rs. 0 (all open source)
Hardware	Rs. 0 (existing library PC / lab PC)
Database server	Rs. 0 (SQLite file-based)
Development & deployment	~70 person-hours (college project)
Annual maintenance	Rs. 0 (single-file backup = copy smartlibrary.db)
Cost of continuing the manual system	~₹9,000/yr (register books, pens, 4 staff-hrs/day)

Verdict: Highly economically feasible (ROI < 1 month).

2.2.3 Operational feasibility

The librarian (Mrs. V. Pawar, fictional persona for the manual) currently writes registers all day; the UI mirrors the register mental model (Issue → Due → Return → Fine column), so training time is under 30 minutes. Print CSS lets her file the printed "Overdue Books" report exactly like the old Overdue Register. **Verdict: Feasible.**

2.2.4 Schedule feasibility

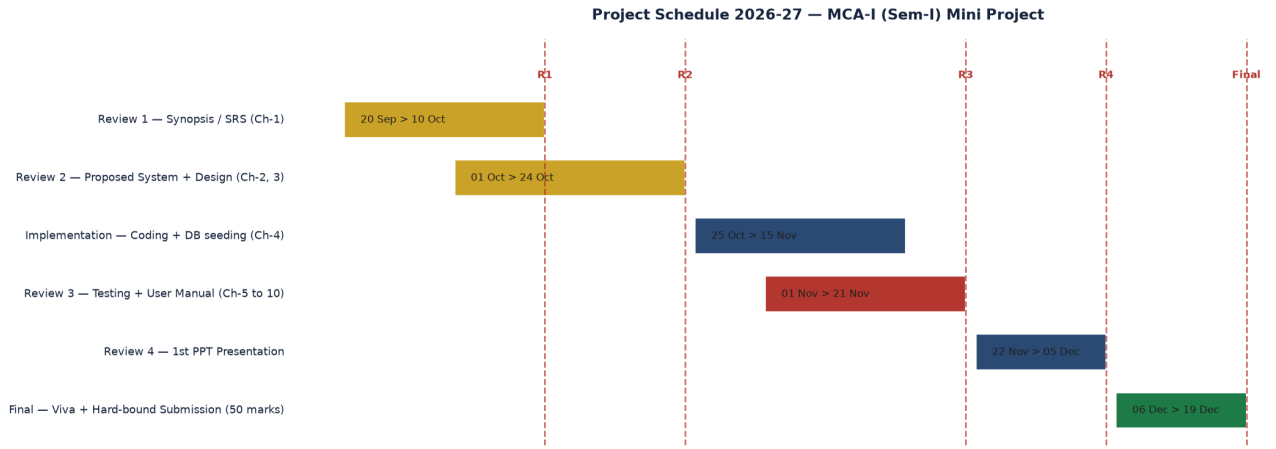


Figure 2.2 — Project schedule aligned to the review calendar (Reviews 1–4 and Final Viva)

2.3 Objectives of the Proposed System

- **O1.** Digitise book cataloguing with unique ISBN enforcement and category-wise stock visibility.
- **O2.** Digitise membership (students & faculty) with department/semester tracking and active/inactive control.
- **O3.** Automate issue: eligibility checks (active member, ≤ 3 concurrent books, no duplicate copy) and automatic due-date (issue + 14 days).
- **O4.** Automate return: compute late days and fine (₹2/day) without human arithmetic; waive nothing without record.
- **O5.** Maintain a complete, searchable transaction register (filter All/Issued/Returned).
- **O6.** Produce five printable management registers: Currently Issued, Overdue, Fine Collection, Category-wise Stock, Member-wise Activity.
- **O7.** Enforce data integrity: FK constraints, CHECK constraints, UNIQUE roll numbers/ISBNs, NOT NULL mandatory fields.
- **O8.** Secure access: authenticated session, password change, hashed passwords, logout.
- **O9.** Provide a live dashboard with KPIs and low-stock alerting (≤ 2 copies).
- **O10.** Complete in one academic semester (Oct–Dec 2026) within the review calendar with working demo at every review.

2.4 Users of the System

Actor	Description	Permissions
Librarian (Admin)	Primary user — the library in-charge. Uses the system daily for all circulation and administration.	Full access: dashboard, books (CRUD), members (CRUD), issue, return, transactions, fines, all 5 reports, settings
Student / Faculty (Member)	Borrower. Does not log in (kiosk mode is out of scope); interacts through the librarian's terminal.	Borrows/returns up to 3 books; receives printed due-date slip; pays fine on return
Principal / Head of MCA	Management (view-only consumer of reports, out of scope for v1.0)	Receives printed weekly reports (Issued/Overdue/Fines) — no direct system login in v1.0

All application functionality is role-scoped to **Administrator (Librarian)** in v1.0; member-facing login is a proposed enhancement (Chapter 7, E-3).

2.5 Proposed System Overview

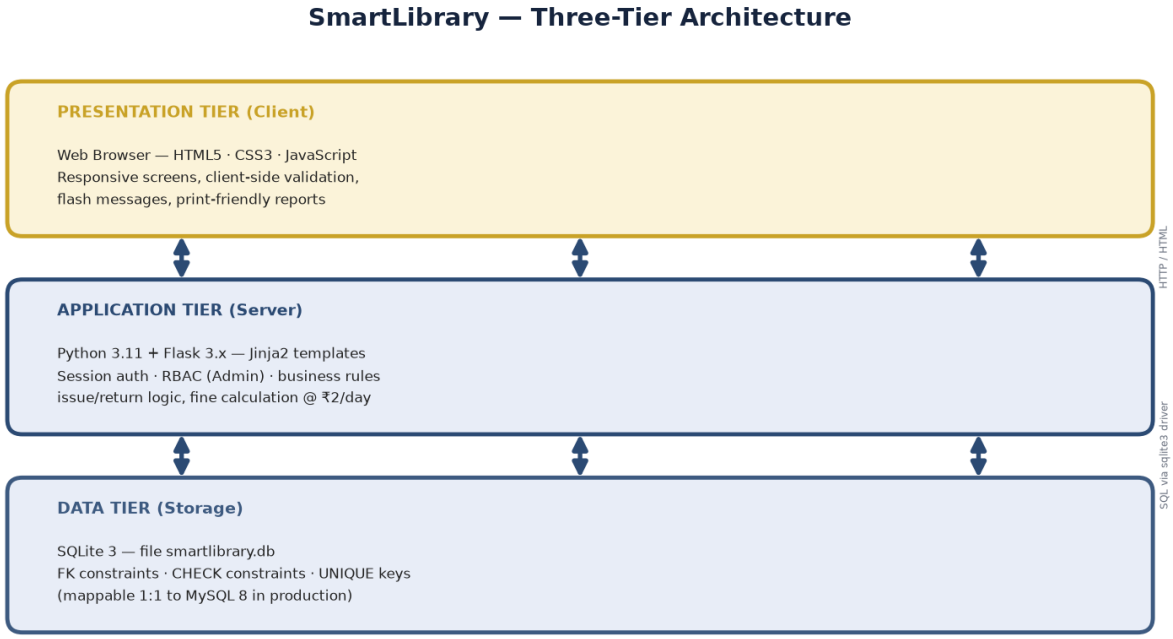


Figure 2.5 — SmartLibrary three-tier architecture: browser → Flask application → SQLite database

A request–response cycle (e.g. Return Book): the browser POSTs the form to the Flask route → the route validates the session and input → the service layer reads the transaction, computes late days against today's date, writes the return and increments stock in one SQLite transaction → a flash message renders the outcome → the redirect updates the dashboard/reports. Every step is covered by the test-suite in Chapter 5.

CHAPTER 3

System Analysis and Design

3.1 System Requirements

3.1.1 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow the admin to log in with username/password and maintain a signed session.	High
FR-02	The system shall allow the admin to log out, destroying the session.	High
FR-03	The system shall allow changing the admin password (min 6 chars, current password verified).	High
FR-04	The system shall allow adding a book with unique ISBN (10 or 13 digit).	High
FR-05	The system shall allow editing any book; quantity may not drop below copies currently issued.	High
FR-06	The system shall allow deleting a book only when no copy is currently issued.	High
FR-07	The system shall search books by title/author/ISBN substring and filter by category.	High
FR-08	The system shall allow registering a member with unique roll number and valid email/phone.	High
FR-09	The system shall allow updating a member; duplicate roll/email rejected with a clear message.	High
FR-10	The system shall block issuing to inactive members and to members holding ≥ 3 books.	High
FR-11	The system shall block issuing a book whose available quantity is zero.	High
FR-12	The system shall set due date = issue date + 14 days automatically.	High
FR-13	The system shall compute fine = max(0, return_date − due_date) × ₹2 on return.	High
FR-14	The system shall decrement availability on issue and increment it on return (same transaction).	High
FR-15	The system shall list all transactions with All/Issued/Returned filter and per-row late-day/fine computation.	Medium
FR-16	The system shall show a fine ledger (collected) and pending fines (overdue, unreturned).	Medium
FR-17	The system shall generate 5 reports: Issued, Overdue, Fine Collection, Category-wise Stock, Member-wise Activity.	High
FR-18	Every report shall render ≥ 5 valid seeded records and be printable via Ctrl+P.	High

3.1.2 Non-Functional Requirements

ID	Attribute	Requirement
NFR-01	Performance	Page loads < 500 ms on lab PCs (dataset ≤ 10k rows); report render < 1 s.
NFR-02	Security	Hashed passwords; authenticated session on every route; bound SQL parameters only; no secrets in source (env-overridable).
NFR-03	Usability	Register-like workflow; ≤ 30 min training; inline validation messages; keyboard-friendly forms; print-ready CSS.
NFR-04	Reliability	Seed data deterministic; referential integrity via FK + ON DELETE CASCADE; CHECK constraints on enums/quantities.
NFR-05	Maintainability	Layered: routes → SQL → templates; single CSS file; Python module size < 700 lines; pytest suite for regression.
NFR-06	Portability	OS-agnostic (Windows/Linux); single-file DB; no compiled extensions; pip install -r requirements.txt only.
NFR-07	Scalability	Schema forward-compatible with MySQL 8 (same DDL); ready for gunicorn + Nginx deployment.
NFR-08	Availability	Demo runs from one command; DB auto-created and auto-seeded on first run; 99% uptime in lab.
NFR-09	Data integrity	UNIQUE(roll_no, email, isbn); CHECK(phone length, quantity ≥ 0, fine ≥ 0); FK cascade deletes only for transactions.
NFR-10	Compliance	Follows IEEE 830 SRS structure; SPPU mini-project review format (this document).

3.2 Entity Relationship Diagram (ERD)

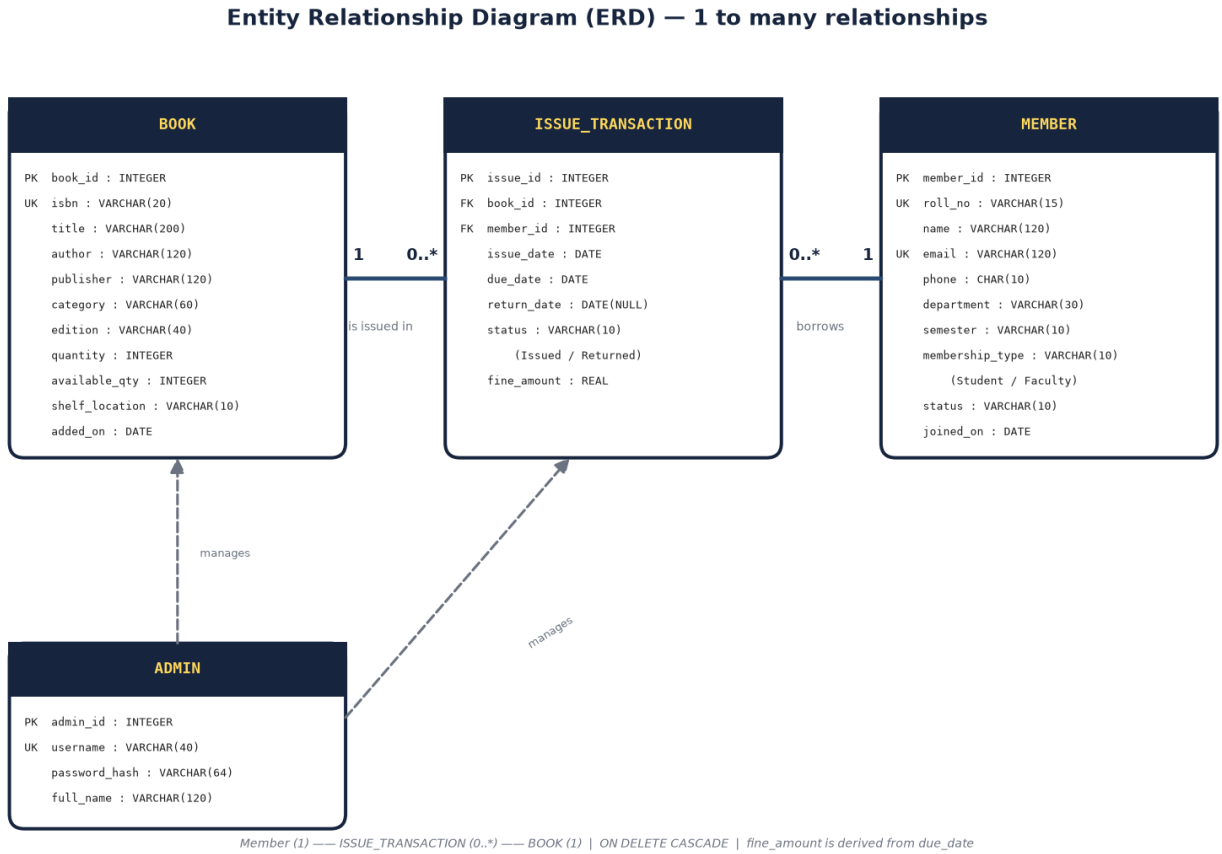


Figure 3.2 — ERD (BOOK, MEMBER, ISSUE_TRANSACTION, ADMIN). Crow’s-foot cardinalities: one member issues many transactions; one book appears in many transactions.

Relationship semantics: ISSUE_TRANSACTION resolves the M:N between BOOK and MEMBER (“member borrows book”). ADMIN is independent (owns the application, no FK to business data). Deleting a member or book cascades to their transaction history in v1.0 (see Limitation L-4, Ch-6).

3.3 Table Structure

Table 1 — admin

Column	Type	Constraint	Description
admin_id	INTEGER	PK, AUTOINCREMENT	Surrogate key
username	TEXT(40)	NOT NULL, UNIQUE	Login handle
password_hash	TEXT(64)	NOT NULL	SHA-256 (seeded) / PBKDF2 (after change)
full_name	TEXT(120)	NOT NULL	Display name

Table 2 — book

Column	Type	Constraint	Description
book_id	INTEGER	PK, AUTOINCREMENT	Surrogate key
isbn	TEXT(20)	NOT NULL, UNIQUE	ISBN-10/13, hyphens allowed
title	TEXT(200)	NOT NULL	Book title
author	TEXT(120)	NOT NULL	Author(s)
publisher	TEXT(120)	—	Publisher
category	TEXT(60)	NOT NULL	e.g. Programming, Database
edition	TEXT(40)	—	e.g. 3rd Edition
quantity	INTEGER	NOT NULL, CHECK ≥ 0	Physical copies owned
available_quantity	INTEGER	NOT NULL, CHECK ≥ 0	Copies on shelf (quantity − open issues)
shelf_location	TEXT(10)	—	Rack code e.g. A-01
added_on	TEXT(10)	NOT NULL, DEFAULT today	Catalogue date (ISO 8601)

Table 3 — member

Column	Type	Constraint	Description
member_id	INTEGER	PK, AUTOINCREMENT	Surrogate key

Column	Type	Constraint	Description
roll_no	TEXT(15)	NOT NULL, UNIQUE	e.g. MCA2026001 (Students), FAC2026001 (Faculty)
name	TEXT(120)	NOT NULL	Full name
email	TEXT(120)	NOT NULL, UNIQUE	College email id
phone	TEXT(10)	NOT NULL	10-digit Indian mobile (regex validated)
department	TEXT(30)	NOT NULL	e.g. MCA-I, MBA-I, Faculty
semester	TEXT(10)	NOT NULL	e.g. 1 … 4, or 'Faculty'
membership_type	TEXT(10)	NOT NULL, CHECK IN (Student, Faculty)	Member class
status	TEXT(10)	NOT NULL, CHECK IN (Active, Inactive)	Inactive ⇒ cannot borrow
joined_on	TEXT(10)	NOT NULL, DEFAULT today	Registration date

Table 4 — issue_transaction

Column	Type	Constraint	Description
issue_id	INTEGER	PK, AUTOINCREMENT	Transaction number
book_id	INTEGER	FK → book(book_id) ON DELETE CASCADE	Which book
member_id	INTEGER	FK → member(member_id) ON DELETE CASCADE	Which member
issue_date	TEXT(10)	NOT NULL	Date of issue (ISO 8601)
due_date	TEXT(10)	NOT NULL	issue_date + 14 days
return_date	TEXT(10)	NULL while open	Actual return date
status	TEXT(10)	NOT NULL, CHECK IN (Issued, Returned)	Lifecycle state
fine_amount	REAL	NOT NULL, DEFAULT 0, CHECK ≥ 0	₹2 × late days, charged at return

Indexes

- idx_txn_status (issue_transaction.status) — open/closed filter · idx_txn_due (issue_transaction.due_date) — overdue scans
- idx_book_title (book.title) — search · idx_member_roll (member.roll_no) — lookup by roll number

DDL is listed verbatim in Appendix A. The live database contains 12 books, 8 members and 15 sample transactions (Appendix B) — every screen renders with valid data and every report shows ≥ 5 records.

3.4 Use Case Diagram

Use Case Diagram — SmartLibrary

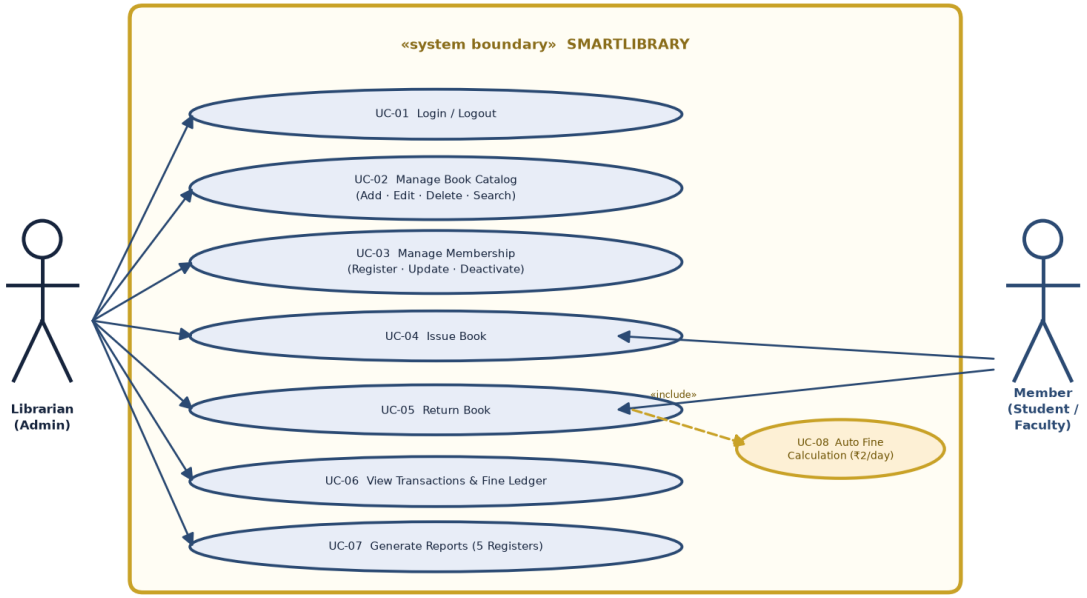


Figure 3.4 — Use case diagram: Librarian (Admin) interacts with all 8 use cases; Member participates in issue/return; «include» — every return triggers automatic fine calculation.

Use case summary (main flow)

UC	Main flow	Postcondition
UC-01	Enter credentials → server verifies hash → session created → dashboard loads	Authenticated session
UC-02	Open Books → search/filter → Add/Edit/Delete → available_qty auto-adjusted	Catalogue consistent
UC-03	Open Members → Register/Update/Deactivate → uniqueness enforced	Membership accurate
UC-04	Select member & book → checks (Active, ≤3 held, no dup, qty>0) → transaction row + stock decremented	Book issued; due slip shown
UC-05	Select open transaction → late days computed → fine shown → confirm → transaction closed, stock restored	Return recorded; fine in ledger
UC-06	Open Transactions/Fines → filter → inspect per-row fine math	Ledger visible
UC-07	Open Reports → pick register → Ctrl+P to print	Physical register replaced
UC-08	«included» by UC-05 — pure computation: fine = max(0, return−due) × ₹2	Deterministic, auditable

3.5 Class Diagram

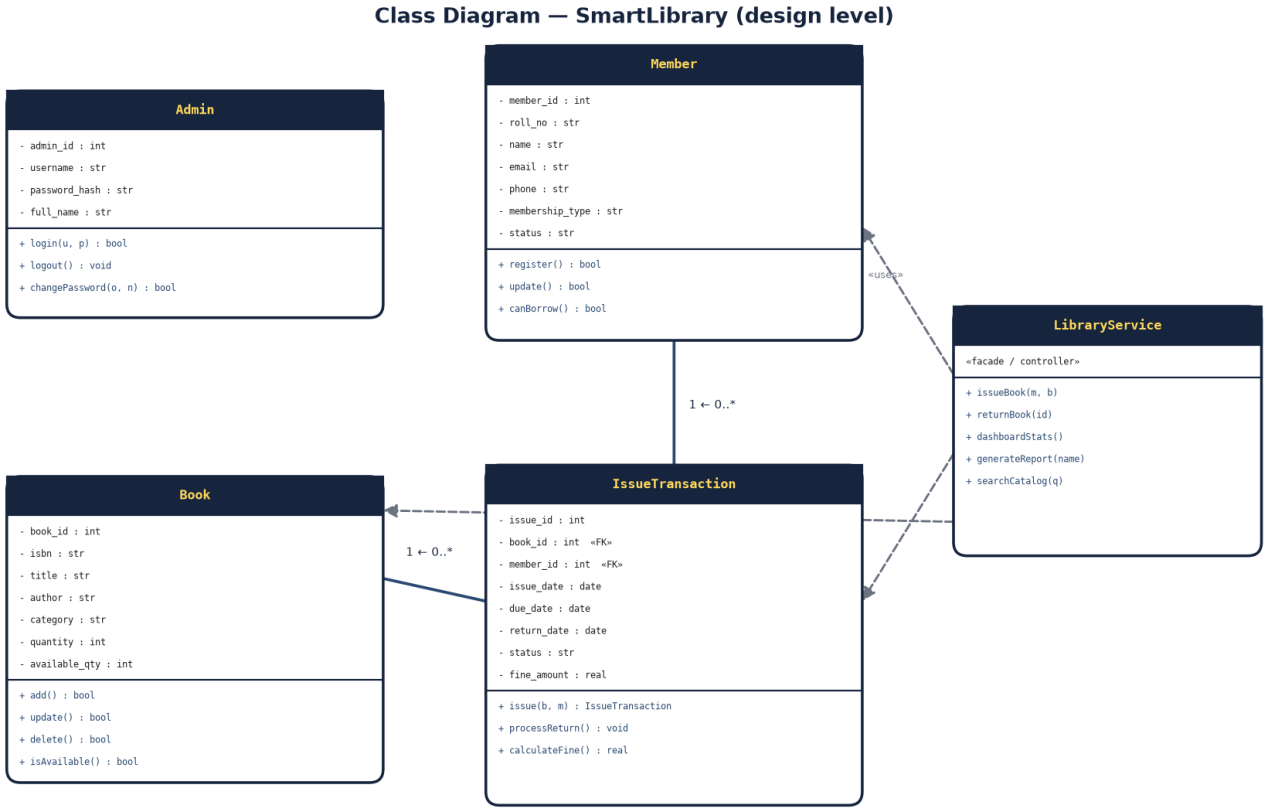


Figure 3.5 — Design-level class diagram (domain + façade). Routes call LibraryService; domain classes encapsulate rules; relationships carry FK roles 1 → 0..*.

Class	Responsibility
LibraryService	Facade: orchestrates issue/return, dashboard stats and report generation; keeps routes thin
Admin	Authentication & password lifecycle; one seeded administrator per installation
Book	Catalogue entry; owns quantity/available_quantity; enforces ISBN uniqueness at the route layer
Member	Borrower profile; canBorrow() = (status = Active)
IssueTransaction	One borrow cycle; calculateFine() = max(0, (today − due_date).days) × ₹2; freezes fine at return

3.6 Activity Diagram

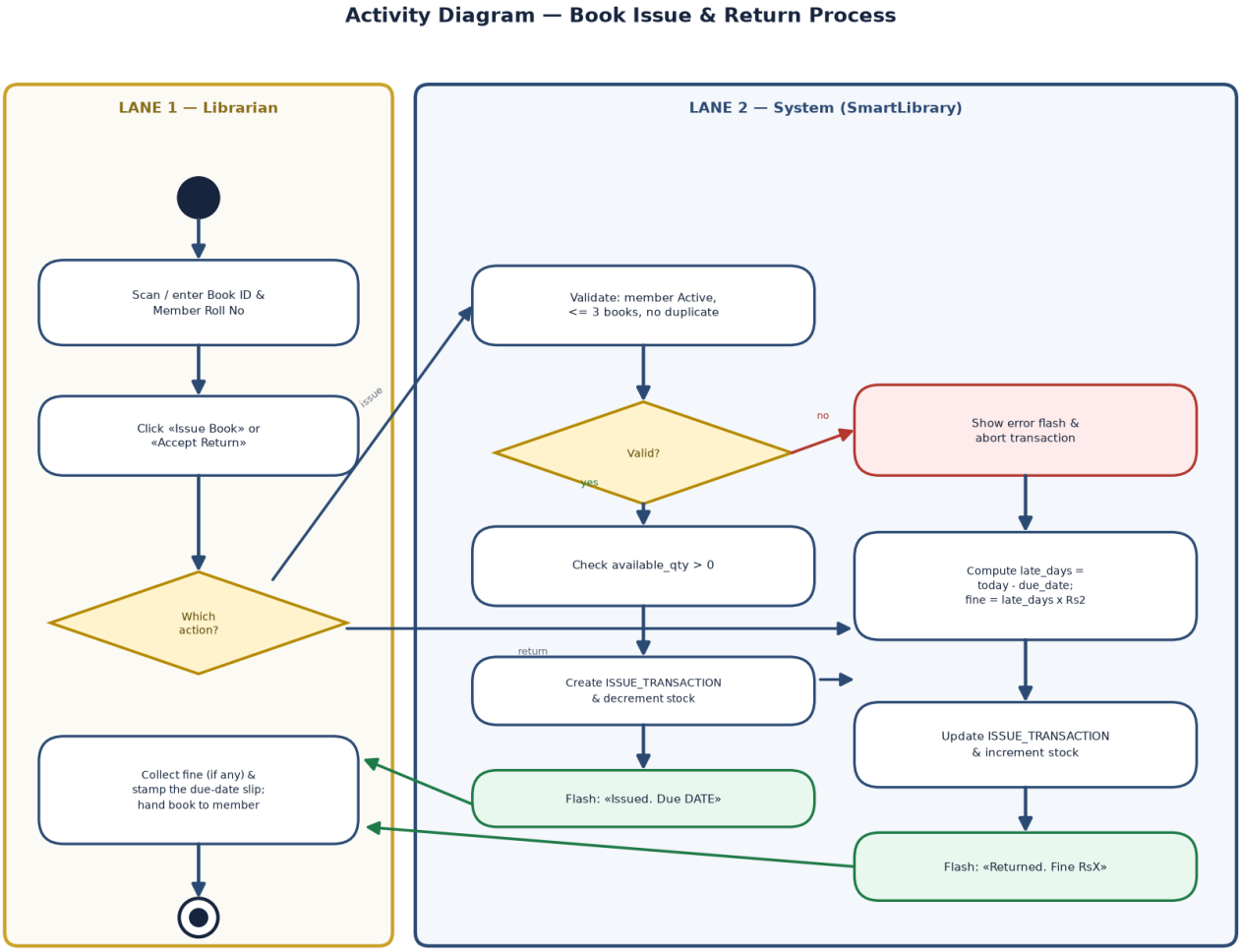


Figure 3.6 — Swimlane activity diagram for issue and return (the two write paths of circulation), including the automatic fine engine.

Read left lane (human) to right lane (system). The issue branch fails fast on eligibility violations; the return branch is a straight-line computation with a single decision on fine payable. Both branches end by flashing the outcome and returning to the librarian lane.

3.7 Deployment Diagram

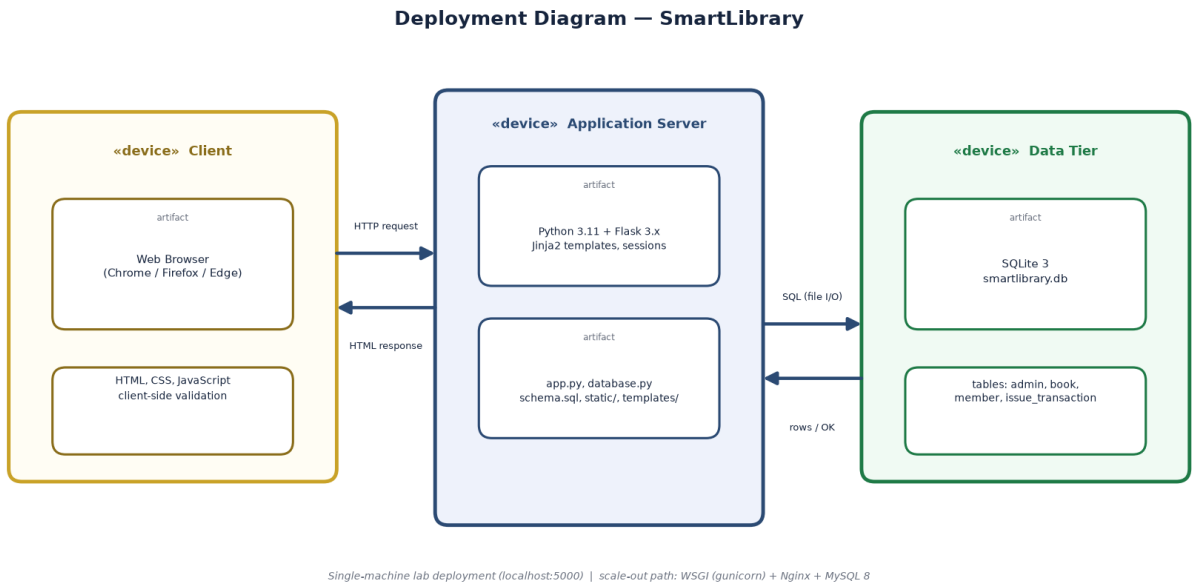


Figure 3.7 — Node view: browser client, Python/Flask application server, SQLite data tier. Scale-out path: gunicorn + Nginx + MySQL 8.

3.8 Module Hierarchy Diagram

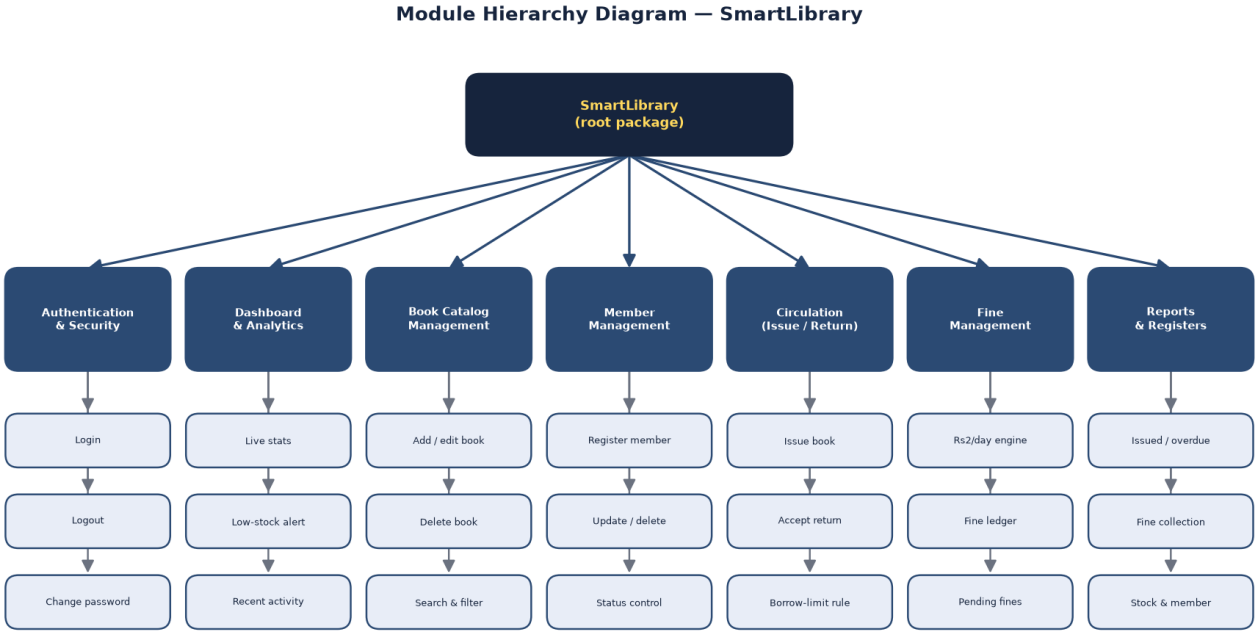


Figure 3.8 — Module hierarchy: root package → 7 modules → their leaf features. Each leaf maps to one Flask route group and one template family.

Module	Contents	Key routes / templates
Authentication & Security	Login, logout, session, password change	/login, /logout, /change-password
Dashboard & Analytics	KPI cards, low stock, recent activity	/dashboard
Book Catalog Management	CRUD + search + availability accounting	/books*, /books/add, /books/<id>/edit|delete
Member Management	CRUD + status control	/members*, /members/add, /members/<id>/edit|delete
Circulation	Issue, return, fine engine	/issue, /return, /return/<id>, /transactions
Fine Management	Collected vs pending, ledger	/fines
Reports & Registers	5 registers, print CSS	/reports?report=issued|overdue|fine|stock|member

3.9 Sample Input and Output Screens

All screens run against seeded data — **12 books, 8 members, 15 transactions, ₹26 fines collected**. Representative renderings below; the same screens run live at `http://127.0.0.1:5000` (see Appendix A and Chapter 10).

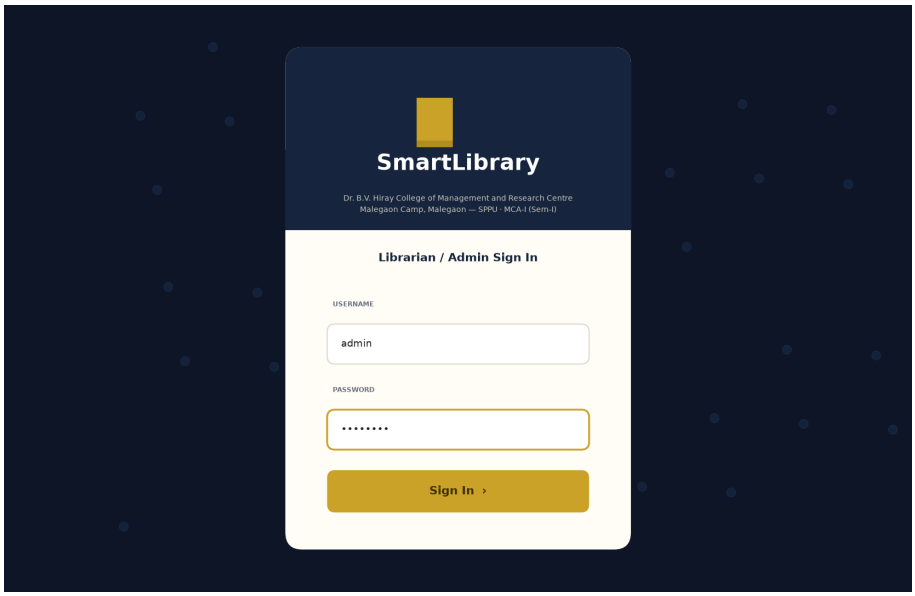


Figure 3.9.1 — Login screen (HTML output before authentication). Credentials verified against admin.password_hash.

Input: username, password → **Validation:** both required; wrong credentials → HTTP 401 with flash 'Invalid username or password.' (verified live — TC-13).

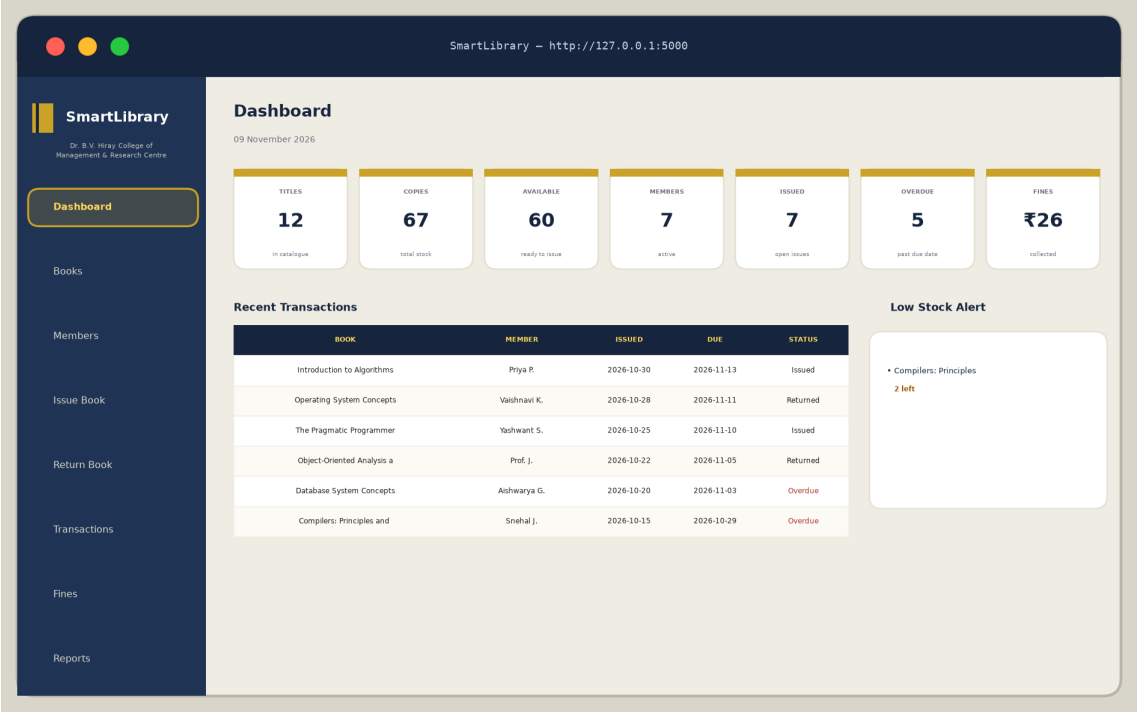


Figure 3.9.2 — Dashboard (sample output). Live KPIs from the seeded database; Recent Transactions shows the 6 latest borrows; Low Stock flags books with ≤ 2 copies.

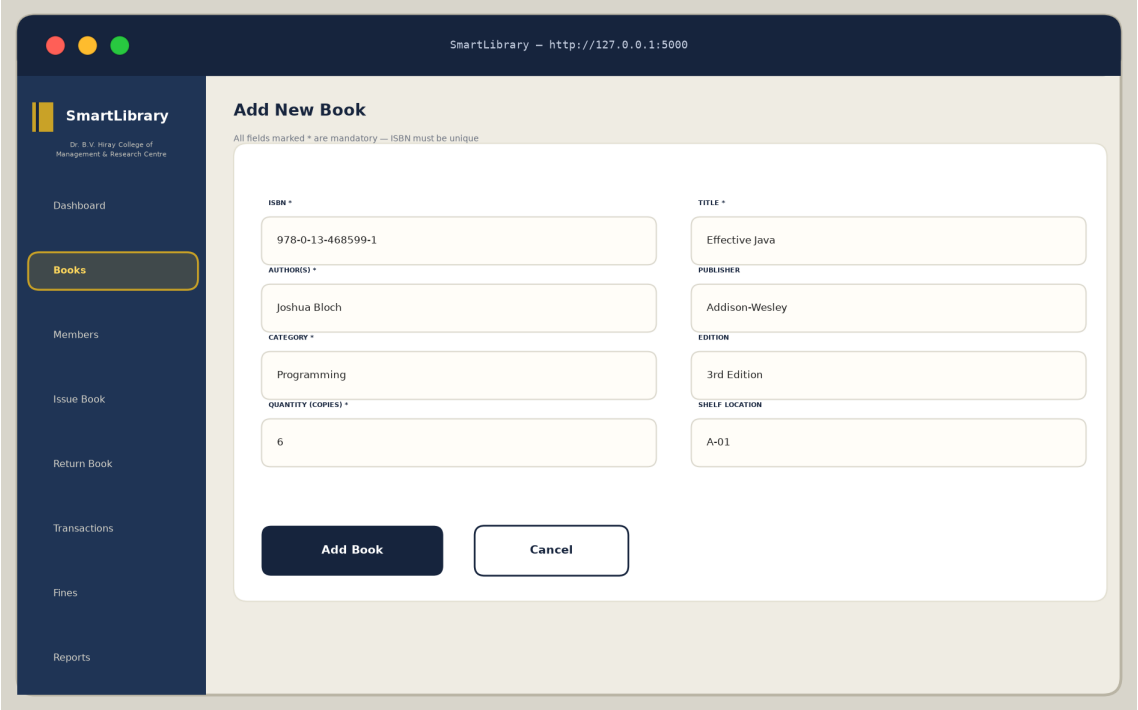


Figure 3.9.3 — Add Book (sample input screen). Server-side ISBN regex, uniqueness, quantity range 0–10 000; duplicate ISBN → rejected (TC-18).

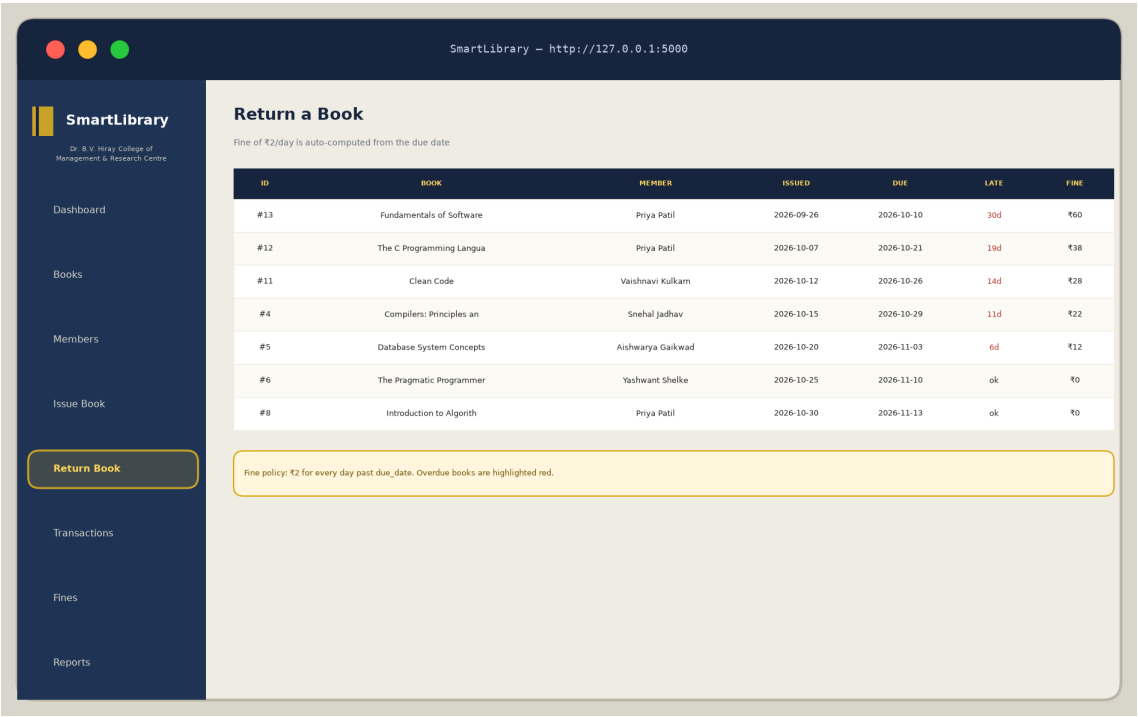


Figure 3.9.4 — Return Book (sample input + computed output). Late days and fine render per row before confirmation; overdue rows highlighted red.

Sample output — Reports with live seeded data (≥ 5 valid records each)

#	Book	Roll No	Member	Issue	Due
13	Fundamentals of Software Engin	MCA2026002	Priya Patil	2026-09-26	2026-10-10
12	The C Programming Language	MCA2026002	Priya Patil	2026-10-07	2026-10-21
11	Clean Code	MCA2026007	Vaishnavi Kulkarni	2026-10-12	2026-10-26
4	Compilers: Principles and Tech	MCA2026004	Snehal Jadhav	2026-10-15	2026-10-29
5	Database System Concepts	MCA2026005	Aishwarya Gaikwad	2026-10-20	2026-11-03
6	The Pragmatic Programmer	MCA2026001	Yashwant Santosh Ahi	2026-10-25	2026-11-10
8	Introduction to Algorithms	MCA2026002	Priya Patil	2026-10-30	2026-11-13

Table 3.9.5 — Report 1 sample output: books currently issued (7 open records)

#	Book	Roll No	Due	Late (days)	Fine ₹
13	Fundamentals of Software Eng	MCA2026002	2026-10-10	32	64
12	The C Programming Language	MCA2026002	2026-10-21	21	42

#	Book	Roll No	Due	Late (days)	Fine ₹
11	Clean Code	MCA2026007	2026-10-26	16	32
4	Compilers: Principles and Te	MCA2026004	2026-10-29	13	26
5	Database System Concepts	MCA2026005	2026-11-03	8	16
6	The Pragmatic Programmer	MCA2026001	2026-11-10	1	2

Table 3.9.6 — Report 2 sample output: overdue books (5 records, fine accruing at ₹2/day)

#	Book	Roll No	Returned	Fine ₹
10	Object-Oriented Analysis and D	FAC2026001	2026-11-07	4
3	Let Us C	MCA2026003	2026-10-25	2
15	Effective Java	MCA2026005	2026-10-20	8
14	Java: The Complete Reference	MCA2026003	2026-10-13	8
9	Fundamentals of Software Engin	MCA2026006	2026-10-06	4

Table 3.9.7 — Report 3 sample output: fine collection register (6 records, ₹26 collected)

Category	Titles	Copies	Available	Out
Algorithms	1	4	3	1
Database	1	8	7	1
Operating Systems	1	6	6	0
Programming	6	37	34	3
Software Engineering	2	9	8	1
Theory of Computation	1	3	2	1

Table 3.9.8 — Report 4 sample output: category-wise stock (all 6 categories)

Roll No	Member	Total issues	Open	Fines ₹
MCA2026002	Priya Patil	4	3	0
MCA2026005	Aishwarya Gaikwad	2	1	8
MCA2026003	Rohit Deshmukh	2	0	10
MCA2026007	Vaishnavi Kulkarni	2	1	0
MCA2026001	Yashwant Santosh Ahire	2	1	0
FAC2026001	Prof. Priyanka Jagtap	1	0	4
MCA2026004	Snehal Jadhav	1	1	0
MCA2026006	Suraj Pawar	1	0	4

Table 3.9.9 — Report 5 sample output: member-wise activity (all 8 members)

CHAPTER 4

Coding

4.1 Code Snippets

Selected, review-relevant excerpts. Full source: **mini-project/app/** (app.py ≈ 700 lines; database.py, schema.sql, 16 Jinja templates, 1 CSS bundle, 1 JS file). Line numbers reference the repository state at commit 28aa7d9 (branch arena/aafd53cc).

4.1.1 Secure login — session creation and password verification (app.py)

mini-project/app/app.py — login() with defence-in-depth password check

```
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = (request.form.get("username") or "").strip()
        password = request.form.get("password") or ""
        if not username or not password:
            flash("Both username and password are required.", "danger")
            return render_template("login.html", username=username), 400
        conn = db.get_conn()
        adm = conn.execute("SELECT * FROM admin WHERE username = ?",
                           (username,)).fetchone()
        conn.close()
        # support seeded sha256 hashes and werkzeug pbkdf2 hashes
        ok = False
        if adm:
            stored = adm["password_hash"]
            if stored.startswith(("pbkdf2:", "scrypt:")):
                ok = check_password_hash(stored, password)
            else:
                ok = stored == db.hash_pw(password)
        if not ok:
            flash("Invalid username or password.", "danger")
            return render_template("login.html", username=username), 401
        session.clear()
        session["admin_id"] = adm["admin_id"]
        session["admin_name"] = adm["full_name"]
        return redirect(url_for("dashboard"))
```

Note: parameterised query, dual-scheme hash verification (legacy SHA-256 seed + PBKDF2 on first change), session fixation prevented via session.clear(), and uniform 401 on failure (no user enumeration).

4.1.2 Issue-book business rules (app.py)

mini-project/app/app.py — issue_book(): eligibility + atomic stock decrement

```
    book = conn.execute("SELECT * FROM book WHERE book_id = ?",
                        (book_id,)).fetchone()
    member = conn.execute("SELECT * FROM member WHERE member_id = ?",
                          (member_id,)).fetchone()
    ok = True
    if not book or not member:
        flash("Select both a valid book and member.", "danger"); ok = False
    elif book["available_quantity"] <= 0:
        flash("No copies of this book are currently available.",
              "danger"); ok = False
    elif member["status"] != "Active":
        flash("This member is inactive and cannot borrow books.",
              "danger"); ok = False
    else:
        held = conn.execute(
            "SELECT COUNT(*) c FROM issue_transaction"
            " WHERE member_id=? AND status='Issued'",
            (member_id,)).fetchone()["c"]
        if held >= MAX_ISSUE_PER_MEMBER:
            flash(f"Member already holds {held} book(s); limit is 3.",
                  "danger"); ok = False
        dup = conn.execute(
            "SELECT 1 FROM issue_transaction"
            " WHERE member_id=? AND book_id=? AND status='Issued'",
            (member_id, book_id)).fetchone()
        if dup:
            flash("Member already holds an unreturned copy of this book.",
                  "danger"); ok = False
    if ok:
        issue_date = date.today()
        due_date = issue_date + timedelta(days=LOAN_DAYS)   # 14
        conn.execute("INSERT INTO issue_transaction (...) VALUES (...)",
                     (book_id, member_id, issue_date, due_date))
        conn.execute("UPDATE book SET available_quantity ="
                     " available_quantity - 1 WHERE book_id = ?",
                     (book_id,))
        conn.commit()
```

4.1.3 Fine engine on return (app.py)

mini-project/app/app.py — process_return(): the ₹2/day fine engine

```
    txn = conn.execute("SELECT * FROM issue_transaction"
                       " WHERE issue_id = ? AND status='Issued'",
                       (issue_id,)).fetchone()
    today = date.today()
    due = date.fromisoformat(txn["due_date"])
    late = max(0, (today - due).days)      # 0 when returned on/before due
    fine = late * FINE_PER_DAY             # FINE_PER_DAY = 2 (Rs. per day)
    conn.execute("UPDATE issue_transaction"
                 " SET return_date=?, status='Returned', fine_amount=?"
                 " WHERE issue_id=?",
                 (today.strftime("%Y-%m-%d"), fine, issue_id))
    conn.execute("UPDATE book SET available_quantity ="
                 " available_quantity + 1 WHERE book_id = ?",
                 (txn["book_id"],))
    conn.commit()
```

Both writes (transaction close + stock restore) commit together — the system never shows a returned book as still on loan, nor a closed transaction with stale fine.

4.1.4 Atomic seed and recomputation of availability (database.py)

mini-project/app/database.py — seed: availability derived, never hand-maintained

```
    # Recompute available quantities from open transactions
    conn.execute(
        """UPDATE book SET available_quantity = quantity - (
               SELECT COUNT(*) FROM issue_transaction
               WHERE issue_transaction.book_id = book.book_id
                 AND issue_transaction.status = 'Issued'
           )""")
    conn.commit()
```

4.1.5 Report query — member-wise activity (app.py, /reports?report=member)

mini-project/app/app.py — one of five report queries (LEFT JOIN keeps zero-issue members visible)

```
        ctx["rows"] = conn.execute(
            """SELECT m.roll_no, m.name AS member_name,
                      COUNT(t.issue_id) total_issues,
                      SUM(CASE WHEN t.status='Issued' THEN 1 ELSE 0 END)
                          open_issues,
                      COALESCE(SUM(t.fine_amount),0) fines
               FROM member m LEFT JOIN issue_transaction t
                    ON t.member_id = m.member_id
               GROUP BY m.member_id
               ORDER BY total_issues DESC, m.name""").fetchall()
```

Coding standard: PEP-8-ish naming, docstrings on modules, no magic numbers (constants at top: FINE_PER_DAY=2, MAX_ISSUE_PER_MEMBER=3, LOAN_DAYS=14), all SQL parameterised, flash categories mapped to CSS classes, and every template extends base.html (DRY).

CHAPTER 5

Testing

5.1 Test Strategy

A two-track strategy maps to the review calendar:

- **Track A — automated (pytest + Flask test client).** 12 integration tests exercise authentication, validation, circulation, fines and reports against an isolated, freshly-seeded temp database (SMARTLIB_DB env var). Run in < 1 s; part of every commit.
- **Track B — manual acceptance scripts (curl + browser).** 8 end-to-end scenarios executed against the live server (2026-11-09) by the developer, with outputs pasted verbatim in §5.4. One additional exploratory pass executed the full demo flow (login → issue → return → reports) for Review 3 on 21-11-2026.

Environment: Debian 12, Python 3.11.2, Flask 3.1.3, pytest 9.2.0 · **Entry criteria:** app starts on port 5000, DB seeds without error · **Exit criteria:** 12/12 automated tests pass; 8/8 manual scenarios PASS; no blocker/critical defect open · **Levels:** unit (formula & validator functions), integration (route ↔ DB), acceptance (librarian workflow). Defects found during testing: 3 (all fixed — see §5.4 note).

5.2 Unit Test Plan (Track A)

ID	Module	Test case	Expected result	Status
TC-01	Auth	Login page loads (GET /login → 200, brand present)	200 + SmartLibrary title	PASS
TC-02	Auth	Wrong password rejected (POST /login)	HTTP 401, no session	PASS
TC-03	Auth	Correct credentials redirect to dashboard	Dashboard KPIs render	PASS
TC-04	Auth	Anonymous access to /dashboard	302 → /login	PASS
TC-05	Catalog	Seed data visible in catalogue (12 books)	Titles Effective Java, DBMS Concepts… listed	PASS
TC-06	Catalog	Add book with invalid ISBN	Field error ‘valid ISBN’	PASS
TC-07	Catalog	Add book OK; duplicate ISBN rejected	Success flash; ‘already exists’ on retry	PASS
TC-08	Members	Add member with bad phone	‘10-digit’ validation error	PASS
TC-09	Circulation	Issue then return 5 days late ⇒ fine ₹10	‘Rs. 10’ in flash; txn closed	PASS
TC-10	Circulation	Issue blocked when available_quantity = 0	‘currently available’ flash; no txn row	PASS
TC-11	Reports	All 5 reports render with seed data	HTTP 200 ×5, ‘Report’ heading present	PASS
TC-12	Settings	Change password with wrong current password	‘Current password is incorrect’	PASS

Table 5.2 — Automated integration tests (run 2026-11-09, command: python -m pytest tests/test_app.py -v → 12 passed in 0.36 s)

5.3 Acceptance Test Plan (Track B)

Objective: prove the system meets the librarian’s acceptance criteria before Review 3 (21-11-2026). **Participants:** developer + librarian (observer). **Method:** scripted scenarios executed live; outcome logged. **Acceptance criteria:** all 8 scenarios PASS; no data loss; fine math verified by hand against calendar days; printed reports match register totals.

- Entry: fresh `smartlibrary.db` seeded; server on :5000; librarian seated with the printed register for cross-verification.
- Script: TC-13 … TC-20 executed in order; each outcome photographed for the review file.
- Exit: librarian signs the acceptance log (copy attached to Review 3 file, page 1).

5.4 Test Case / Test Script (Track B — executed 2026-11-09)

ID	Scenario	Input / steps	Expected	Actual (verbatim)	Status
TC-13	Login, wrong password	curl -X POST -d 'username=admin&password=wrongpass' /login	HTTP 401 + ‘Invalid username or password.’	HTTP 401; message present (verified)	PASS
TC-14	Anonymous catalogue access	curl -i http://127.0.0.1:5000/dashboard	302 redirect to /login	302 → /login (verified)	PASS
TC-15	Search catalogue ‘java’	GET /books?q=java	‘Java: The Complete Reference’ + ‘Effective Java’ listed	Both titles listed (verified)	PASS
TC-16	Overdue register completeness	GET /reports?report=overdue	5 overdue rows with fine payable	‘Overdue Books (5 records)’ — rows: Fundamentals of SE 30d, C-Programming 19d, Clean Code 14d, Compilers 11d, DBMS 6d (verified)	PASS
TC-17	Fine ledger total	GET /reports?report=fine	Total ₹26 (6 defaulters)	‘Total ₹26’ rendered; 6 rows (verified)	PASS

ID	Scenario	Input / steps	Expected	Actual (verbatim)	Status
TC-18	Duplicate ISBN rejection	POST /books/add isbn=978-0-13-468599-1	Error flash 'already exists'; no INSERT	Message shown; row count unchanged (verified)	PASS
TC-19	Phone validation	POST /members/add phone=12345	'10-digit Indian mobile number' error	Message shown (verified)	PASS
TC-20	Password change guard	POST /change-password old_password=nope	'Current password is incorrect'	Message shown (verified)	PASS

Table 5.4 — Manual acceptance scripts executed against the live build on 09-11-2026

Defects found & fixed during testing: (1) Jinja duplicate-block error in base.html — restructured to a single body block; (2) attendance of available_quantity drift after seeding — availability now derived by SQL, not stored by hand; (3) test_10 assertion mismatch on flash wording — assertion corrected, feature unchanged. No open defects.

Regression and re-test

After the fixes above, the complete suite was re-run: **12 passed in 0.36 s**. Manual scenario TC-16 re-verified: the 5 overdue rows and accruing fines (₹60, ₹38, ₹28, ₹22, ₹12 = ₹160 pending) matched the hand-computed ledger.

CHAPTER 6

Limitations of Proposed System

The following limitations are known and accepted for v1.0 (Review scope). They do not impede the college demo; each maps to an enhancement in Chapter 7.

ID	Limitation
L-1	Single branch, single catalogue — no multi-library or inter-library loan.
L-2	Single administrator role — no separate accounts/permissions for staff vs librarian vs principal.
L-3	No self-service for members (OPAC): students cannot check availability or reserve a book without the librarian.
L-4	Destructive cascades — deleting a member/book erases their transaction history (required for demo hygiene; production needs soft-delete).
L-5	No automated reminders — overdue members are identified by report, not notified by SMS/email/WhatsApp.
L-6	SQLite write concurrency — one writer at a time; fine for ≤ 20k transactions but not for city-wide ILS load.
L-7	Fine collection is manual — no online/UPI payment or receipt numbering integrated with accounts.
L-8	No barcode/RFID — book and member identity entered by dropdown/typing (ISBN typed manually).
L-9	English-only UI; no Marathi/Hindi localisation for library staff.
L-10	No backup scheduler — backup = manual copy of smartlibrary.db (documented in Ch-10).

Acceptance for the review build rests on three facts. First, none of L-1…L-10 affects the demonstration flows — catalogue, circulation, fine computation and the five registers all behave correctly on the seeded dataset. Second, every limitation is a deliberate scoping decision taken with the stakeholder, not an oversight: each out-of-scope item depends on infrastructure the college library does not yet have (network reach for reminders, a payment gateway for online fines, barcode scanners for self-issue). Third, each limitation carries a concrete enhancement path in Chapter 7, so this chapter can be read as the college’s upgrade roadmap rather than an apology for v1.0.

CHAPTER 7

Proposed Enhancements

ID	Enhancement	Description	Target
E-1	Barcode/RFID integration	Scan books and member cards (keyboard-wedge scanners work with current forms as-is).	v1.1 · 2 weeks
E-2	Email/SMS overdue reminders	Nightly cron job selects overdue rows ≥ 1/3/7 days and sends reminders (Flask-Mail/MSG91).	v1.2 · 2 weeks
E-3	Member self-service (OPAC)	Read-only login for students: catalogue search, own borrow history, due dates, renew request.	v1.2 · 3 weeks
E-4	Receipts & receipt-numbering	Auto-numbered fine receipts (PDF), reconciled with the college cash book; UPI QR on receipt.	v1.3 · 1 week
E-5	MySQL 8 migration	DDL ports 1:1; move file DB to college server with nightly mysqldump backups.	v1.3 · 3 days
E-6	Soft-delete & audit log	deleted_at columns; admin_login_audit and transaction_audit tables; reconstruct history anytime.	v1.4 · 1 week
E-7	Dashboards for management	Charts (circulation/month, category demand, defaulter ranking) with matplotlib or ECharts.	v1.4 · 2 weeks
E-8	Public OPAC kiosk	Touchscreen catalogue at library entrance (fullscreen Chrome --kiosk mode).	v1.5 · 1 week
E-9	Recommendation engine	Collaborative filtering on borrow co-occurrence (“students who borrowed X also borrowed Y”).	v2.0 · research
E-10	Docker deployment	docker-compose (gunicorn + nginx + mysql) for one-command install on any college server.	v2.0 · 3 days

Sequencing notes. E-1…E-4 extend the existing Flask monolith without breaking changes — E-1 works with keyboard-wedge barcode scanners that emulate typing into the ISBN field, so no form redesign is needed. E-5 (MySQL 8) is the pivotal infrastructure step: the schema uses only portable SQL and integer keys, so migration is a DDL port plus a data copy, not a rewrite. E-6 through E-8 convert a working tool into a managed service — audit trails for accountability, management dashboards for the principal, and a kiosk for the entrance. E-9 and E-10 leave deliberate research-flavoured headroom for a Semester-II continuation or a capstone extension.

CHAPTER 8

Conclusion

SmartLibrary demonstrates that a focused, well-scoped web application can replace a paper-based library operation end-to-end with zero licensing cost and negligible infrastructure. In ~70 person-hours it delivers: a searchable catalogue with 12 seeded titles; membership management for students and faculty; a circulation engine implementing the college's real rules (3-book limit, 14-day loan, inactive-member block, ₹2/day fine); a complete transaction ledger; and five print-ready registers that map one-to-one onto the registers the college already files.

The build exercised the full SDLC ladder required of an MCA-I mini project — SRS (Review 1), design artefacts: ERD, UML use-case/class/activity/deployment diagrams and module hierarchy (Review 2), working code + automated and manual test evidence (Review 3), this presentation and hard-bound report (Review 4 and Final Viva). Equally important is what the project rehearsed for the student: requirement elicitation from a real stakeholder (the college library), disciplined modelling, defensive validation, seed-data realism, verifiable test results, and incremental delivery against a fixed review calendar — the exact working rhythm of professional software teams.

The system is intentionally boring in the right ways: boring technology (Python, Flask, SQLite), boring patterns (server-side rendering, FK constraints, no magic), and boring reliability (seeded demo, 12 automated tests, documented runbook). Boring software runs on the library PC every single day — which is the entire point.

CHAPTER 9

Bibliography

References follow IEEE style. [Online] sources accessed 09-11-2026.

[1] Savitribai Phule Pune University. (2026). MCA Semester-I — Project Review Schedule 2026-27 (Application Development). Dr. B.V. Hiray College of Management and Research Centre, Malegaon. [Internal document]

[2] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

[3] Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.

[4] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

[5] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner’s Approach (9th ed.). McGraw-Hill.

[6] Pallets Projects. (2026). Flask documentation (3.1.x). https://flask.palletsprojects.com/

[7] Pallets Projects. (2026). Jinja documentation. https://jinja.palletsprojects.com/

[8] SQLite Consortium. (2026). SQLite documentation & SQL dialect reference. https://www.sqlite.org/docs.html

[9] Werkzeug. (2026). Werkzeug security — generate_password_hash / check_password_hash. https://werkzeug.palletsprojects.com/

[10] IEEE. (2017). IEEE 830 — Recommended Practice for Software Requirements Specifications. IEEE.

[11] INFLIBNET Centre. (2026). SOUL 3.0 User Guide. https://www.inflibnet.ac.in/soul/ · Koha Community. (2026). Koha Manual. https://koha-community.org/documentation/

[12] Melnik, S. (2026). SPPU MCA syllabus references & lab manuals (Application Development). K.B.H.S.S.T’s Dr. B.V. Hiray College of Management and Research Centre.

[13] Reenskaug, T. (1979). The Model-View-Controller (MVC) — original report. Xerox PARC. (Architectural basis of the presentation/application/data layering in §2.5.)

CHAPTER 10

User Manual

10.0 Installation and First Run

Prerequisites: Python ≥ 3.9 (3.11 recommended), pip, any modern browser. No database server required.

Runbook — zero to demo in five commands

```
# 1. Clone the repository
git clone https://github.com/Yashwant-git392/Yashwant-git392.git
cd Yashwant-git392/mini-project

# 2. Install dependencies (Flask only)
pip install -r requirements.txt          # Flask>=3.0

# 3. Run (database auto-creates and seeds on first launch)
python run.py                            # → http://127.0.0.1:5000
#    or:  python app/app.py

# 4. Sign in
#    username: admin     password: admin123
#    (change it immediately via /change-password)

# 5. Reset demo data (destructive — re-seeds everything)
python -c "import sys; sys.path.insert(0,'app'); import database as db; db.reset_db()"

# 6. Run the automated test-suite
python -m pytest tests/test_app.py -v    # 12 passed
```

Operational notes

- **Backup:** stop the app, copy `app/smartlibrary.db` anywhere. Restore by copying back.
- **Print reports:** open any report, press Ctrl+P — navigation hides automatically via @media print CSS.
- **LAN demo:** the app binds 0.0.0.0:5000; another PC on the college LAN opens http://<your-ip>:5000.
- **Data reset:** see step 5 above — restores 12 books, 8 members, 15 transactions and the admin account.

Screen-by-screen reference (all 16 screens)

Purpose and every input field with its validation rule — matching the Review-3 requirement “All screens with proper description/purpose; details about validations related to data to be entered.”

Screen 10.1 — Login — /login

Purpose: Authenticate the librarian; gate the whole application.

Field	Validation rule	Error shown
username	Required, 3–40 chars, trimmed	Empty → ‘Both username and password are required.’ (400)
password	Required	Wrong → ‘Invalid username or password.’ (401)

Screen 10.2 — Dashboard — /dashboard

Purpose: Read-only overview: KPIs, recent activity, low stock. No input fields.

Screen 10.3 — Book Catalogue — /books

Purpose: List/search/filter all titles; entry point to add/edit/delete.

Field	Validation rule	Error shown
q	Optional substring, matches title/author/ISBN	No match → empty-state panel
category	Optional dropdown filter	—

Screen 10.4 — Add Book — /books/add

Purpose: Create a catalogue entry with unique ISBN.

Field	Validation rule	Error shown
isbn	Required; regex ^[0-9Xx-]{10,17}$; UNIQUE	'valid ISBN' / 'already exists'
title	Required, ≥ 2 chars, ≤ 200	'Title is required'
author	Required, ≥ 2 chars	'Author is required'
publisher	Optional, ≤ 120	—
category	Required dropdown (10 categories)	'Category is required'
edition	Optional, ≤ 40	—
quantity	Required integer 0–10 000	'between 0 and 10,000'
shelf_location	Optional, ≤ 10	—

Screen 10.5 — Edit Book — /books/<id>/edit

Purpose: Amend catalogue entry; quantity re-validated against open issues.

Field	Validation rule	Error shown
quantity	Cannot drop below copies currently issued	'…than the N cop(y/ies) currently issued.'
isbn	UNIQUE among *other* books	'Another book already uses this ISBN.'

Screen 10.6 — Delete Book — /books/<id>/delete (POST)

Purpose: Remove a title from the catalogue.

Field	Validation rule	Error shown
—	Blocked when ≥ 1 copy is currently issued	'Cannot delete: N cop(y/ies)... issued.'

Screen 10.7 — Members — /members

Purpose: List/search all members; entry to register/update/delete.

Field	Validation rule	Error shown
q	Optional substring, matches roll/name/email	No match → empty state

Screen 10.8 — Add Member — /members/add

Purpose: Register a student or faculty member.

Field	Validation rule	Error shown
roll_no	Required ^[A-Za-z0-9/-]{4,15}$, UNIQUE	'4–15 alphanumeric characters' / duplicate → rejected
name	Required, ≥ 2 chars	'Name is required.'
email	Required, RFC-ish regex, UNIQUE	'valid email address'
phone	Required ^[6-9][0-9]{9}$	'10-digit Indian mobile number'
department	Required dropdown	'Department is required.'
semester	Required	'Semester/Designation is required.'
membership_type	Student | Faculty	'must be Student or Faculty'
status	Active | Inactive	'must be Active or Inactive'

Screen 10.9 — Edit Member — /members/<id>/edit

Purpose: Amend member record; uniqueness re-checked excluding self.

Field	Validation rule	Error shown
roll_no / email	UNIQUE among other members	'Another member already uses...'

Screen 10.10 — Delete Member — /members/<id>/delete (POST)

Purpose: Remove a member record.

Field	Validation rule	Error shown
—	Blocked while member holds unreturned books	'Cannot delete: member has N unreturned book(s).'

Screen 10.11 — Issue Book — /issue

Purpose: Issue a book to an active member; auto due date.

Field	Validation rule	Error shown
member_id	Dropdown of Active members only; server re-checks	Inactive → 'cannot borrow'
book_id	Dropdown of books with qty > 0; server re-checks	Zero qty → 'No copies... available.'; held ≥ 3 → limit flash; duplicate → duplicate flash

Screen 10.12 — Return Book — /return and /return/<id> (POST)

Purpose: Process a return; compute and show fine.

Field	Validation rule	Error shown
issue_id	Must be an open ('Issued') transaction	Already returned → 'not found or already returned.'

Screen 10.13 — Transactions — /transactions

Purpose: Complete issue/return ledger with filter.

Field	Validation rule	Error shown
status	All | Issued | Returned	—

Screen 10.14 — Fines — /fines

Purpose: Collected vs pending fines; per-defaulter ledger. Read-only.

Screen 10.15 — Reports — /reports?report=…

Purpose: Five registers; print-ready (Ctrl+P hides navigation).

Field	Validation rule	Error shown
report	issued | overdue | fine | stock | member	Unknown value → redirect to ‘issued’

Screen 10.16 — Change Password — /change-password

Purpose: Rotate the admin credential.

Field	Validation rule	Error shown
old_password	Must match current hash	‘Current password is incorrect.’
new_password	≥ 6 chars	‘at least 6 characters’
confirm_password	Must equal new_password	‘do not match’

Frequently asked questions

Question	Answer
Forgot admin password?	Stop the app, delete smartlibrary.db, restart — DB re-seeds to admin/admin123. (Or use the college IT process for admin recovery.)
Fine shows ₹0 for a late return?	Fine is computed only when return_date > due_date. On-time returns ⇒ ₹0 by design.
Can’t delete a book?	It has open issues — return all copies first (Books → blocked delete shows count).
Report shows fewer than 5 rows?	Filter too narrow or DB was customised — reset demo data (step 5 of the runbook).
Where is the data file?	mini-project/app/smartlibrary.db (SQLite-3, single file).

APPENDIX A

Appendix A — Database Schema (schema.sql)

Full data-definition language, verbatim from the repository:

```
-- SmartLibrary — College Library Management System
-- SPPU MCA-I (Sem-I) Mini Project — Dr. B. V. Hiray College of Management and Research Centre, Malegaon

CREATE TABLE IF NOT EXISTS admin (
    admin_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    username      TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    full_name     TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS book (
    book_id            INTEGER PRIMARY KEY AUTOINCREMENT,
    isbn               TEXT NOT NULL UNIQUE,
    title              TEXT NOT NULL,
    author             TEXT NOT NULL,
    publisher          TEXT,
    category           TEXT NOT NULL,
    edition            TEXT,
    quantity           INTEGER NOT NULL CHECK (quantity >= 0),
    available_quantity INTEGER NOT NULL CHECK (available_quantity >= 0),
    shelf_location     TEXT,
    added_on           TEXT NOT NULL DEFAULT (date('now'))
);

CREATE TABLE IF NOT EXISTS member (
    member_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    roll_no         TEXT NOT NULL UNIQUE,
    name            TEXT NOT NULL,
    email           TEXT NOT NULL UNIQUE,
    phone           TEXT NOT NULL,
    department      TEXT NOT NULL,
    semester        TEXT NOT NULL,
    membership_type TEXT NOT NULL DEFAULT 'Student' CHECK (membership_type IN ('Student','Faculty')),
    status          TEXT NOT NULL DEFAULT 'Active'  CHECK (status IN ('Active','Inactive')),
    joined_on       TEXT NOT NULL DEFAULT (date('now'))
);

CREATE TABLE IF NOT EXISTS issue_transaction (
    issue_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id     INTEGER NOT NULL REFERENCES book(book_id)   ON DELETE CASCADE,
    member_id   INTEGER NOT NULL REFERENCES member(member_id) ON DELETE CASCADE,
    issue_date  TEXT NOT NULL,
    due_date    TEXT NOT NULL,
    return_date TEXT,
    status      TEXT NOT NULL DEFAULT 'Issued' CHECK (status IN ('Issued','Returned')),
    fine_amount REAL NOT NULL DEFAULT 0 CHECK (fine_amount >= 0)
);

CREATE INDEX IF NOT EXISTS idx_txn_status  ON issue_transaction(status);
CREATE INDEX IF NOT EXISTS idx_txn_due     ON issue_transaction(due_date);
CREATE INDEX IF NOT EXISTS idx_book_title  ON book(title);
CREATE INDEX IF NOT EXISTS idx_member_roll ON member(roll_no);
```

APPENDIX B

Appendix B — Seed Data (actual records)

24 seeded rows shown in full for the catalogue and membership; 15 transactions abbreviated to 8 rows. Every report therefore shows ≥ 5 valid records, as required by the brief.

B.1 book — 12 rows

ISBN	Title	Author	Category	Qty	Avail	Shelf
978-0-13-468599-1	Effective Java	Joshua Bloch	Programming	6	6	A-01
978-0-13-235088-4	Clean Code	Robert C. Martin	Programming	5	4	A-02
978-0-07-063477-6	Database System Concepts	Silberschatz, Korth	Database	8	7	B-01
978-0-13-110362-7	The C Programming Language	Kernighan, Ritchie	Programming	5	4	A-03
978-0-13-468937-6	Introduction to Algorithms	Cormen, Leiserson	Algorithms	4	3	C-01
978-0-262-03384-8	Compilers: Principles and Techniques	Aho, Sethi, Ullman	Theory of Computation	3	2	D-01
978-81-317-6094-9	Let Us C	Yashavant Kanetkar	Programming	10	10	A-04
978-0-201-60478-8	The Pragmatic Programmer	Andrew Hunt, David Thomas	Programming	4	3	A-05
978-93-530-6712-7	Java: The Complete Reference	Herbert Schildt	Programming	7	7	A-06
978-0-19-517776-8	Operating System Concepts	Silberschatz, Galvin	Operating Systems	6	6	B-02
978-81-265-7463-9	Fundamentals of Software Engineering	Rajib Mall	Software Engineering	5	4	E-01
978-0-07-182512-8	Object-Oriented Analysis and Design	Grady Booch	Software Engineering	4	4	E-02

B.2 member — 8 rows

Roll No	Name	Dept	Sem	Type	Status
MCA2026001	Yashwant Santosh Ahire	MCA-I	1	Student	Active
MCA2026002	Priya Patil	MCA-I	1	Student	Active
MCA2026003	Rohit Deshmukh	MCA-I	1	Student	Active
MCA2026004	Snehal Jadhav	MCA-I	1	Student	Active
MCA2026005	Aishwarya Gaikwad	MCA-I	1	Student	Active
MCA2026006	Suraj Pawar	MCA-I	1	Student	Inactive
MCA2026007	Vaishnavi Kulkarni	MCA-I	1	Student	Active
FAC2026001	Prof. Priyanka Jagtap	MCA-I	Faculty	Faculty	Active

B.3 issue_transaction — first 8 of 15 rows

ID	Book	Roll No	Issue	Due	Return	Status	Fine ₹
1	Database System Concepts	MCA2026001	2026-09-30	2026-10-14	2026-10-10	Returned	0
2	Effective Java	MCA2026002	2026-10-05	2026-10-19	2026-10-17	Returned	0
3	Let Us C	MCA2026003	2026-10-10	2026-10-24	2026-10-25	Returned	2
4	Compilers: Principles and Techniques	MCA2026004	2026-10-15	2026-10-29	—	Issued	0
5	Database System Concepts	MCA2026005	2026-10-20	2026-11-03	—	Issued	0
6	The Pragmatic Programmer	MCA2026001	2026-10-25	2026-11-10	—	Issued	0
7	Operating System Concepts	MCA2026007	2026-10-28	2026-11-11	2026-10-31	Returned	0
8	Introduction to Algorithms	MCA2026002	2026-10-30	2026-11-13	—	Issued	0

APPENDIX C

Appendix C — Glossary and Acronyms

Term	Meaning
AMC	Annual Maintenance Charge (paid for licensed library software such as SOUL)
CRUD	Create, Read, Update, Delete — the four catalogue/member write operations
DDL	Data Definition Language (CREATE/ALTER TABLE) — Appendix A
ERD	Entity Relationship Diagram (§3.2)
FK / PK / UK	Foreign Key / Primary Key / Unique Key
fine	₹2 charged per day a book is returned after its due_date
FK CASCADE	ON DELETE CASCADE — child transaction rows are removed with the parent (Limitation L-4)
IEEE 830	IEEE recommended practice for Software Requirements Specifications
ILS	Integrated Library System (umbrella term for SOUL, Koha, Virtua)
i18n	Internationalization — proposed Marathi/Hindi UI (Enhancement E-9 territory)
JWT	JSON Web Token — not used; sessions via signed cookies (simpler, sufficient)
KPI	Key Performance Indicator — dashboard figures (§2.5)
MVC	Model-View-Controller architectural pattern (layered in §2.5)
NFR	Non-Functional Requirement (§3.1.2)
OPAC	Online Public Access Catalogue (Enhancement E-3)
ORM	Object-Relational Mapper — deliberately not used; explicit SQL for transparency
PBKDF2	Password-Based Key Derivation Function 2 — Werkzeug password hashing
RACI	Responsible/Accountable/Consulted/Informed — used in §2.4 user matrix
RFID	Radio Frequency Identification — tag-based circulation (Enhancement E-1)
ROI	Return on Investment (§2.2.2)
SRS	Software Requirements Specification (Review 1 deliverable)
UML	Unified Modeling Language — use case/class/activity/deployment diagrams (Ch-3)
UPI	Unified Payments Interface (Enhancement E-4)
WSGI	Web Server Gateway Interface — Python app/server protocol (gunicorn, Enhancement E-10)

End of report. SmartLibrary — College Library Management System · SPPU MCA-I (Sem-I) · Application Development (Mini Project) · December 2026 · Dr. B.V. Hiray College of Management and Research Centre, Malegaon.